CSE370:DATABASE SYSTEMS

BRAC UNIVERSITY
Inspiring Excellence

Lecture 10: Concurrency Control

LOCK-BASED CONCURRENCY CONTROL

Basic Idea:

- A lock controls access to data items.
- A transaction must get a lock before read/write - meaning before a transaction uses any data, it must ask for permission.
- Only transactions that are compatible together, can run at the same time.

Types of Locks:

- Shared Lock (S-lock): Used for reading only. Multiple transactions can hold this S-lock and read simultaneously.
- Exclusive Lock (X-lock): Used for both read/write. This lock cannot be held simultaneously on the same data item. Only one transaction can read/write. Blocks all other transaction from reading and writing.

	S	X
S	✓	✗
X	✗	✗

Lock Compatibility Matrix

TWO-PHASE LOCKING (2PL)

Growing Phase: In this phase, the transaction collects all the locks it needs to access data. It is not allowed to release any lock during this time.

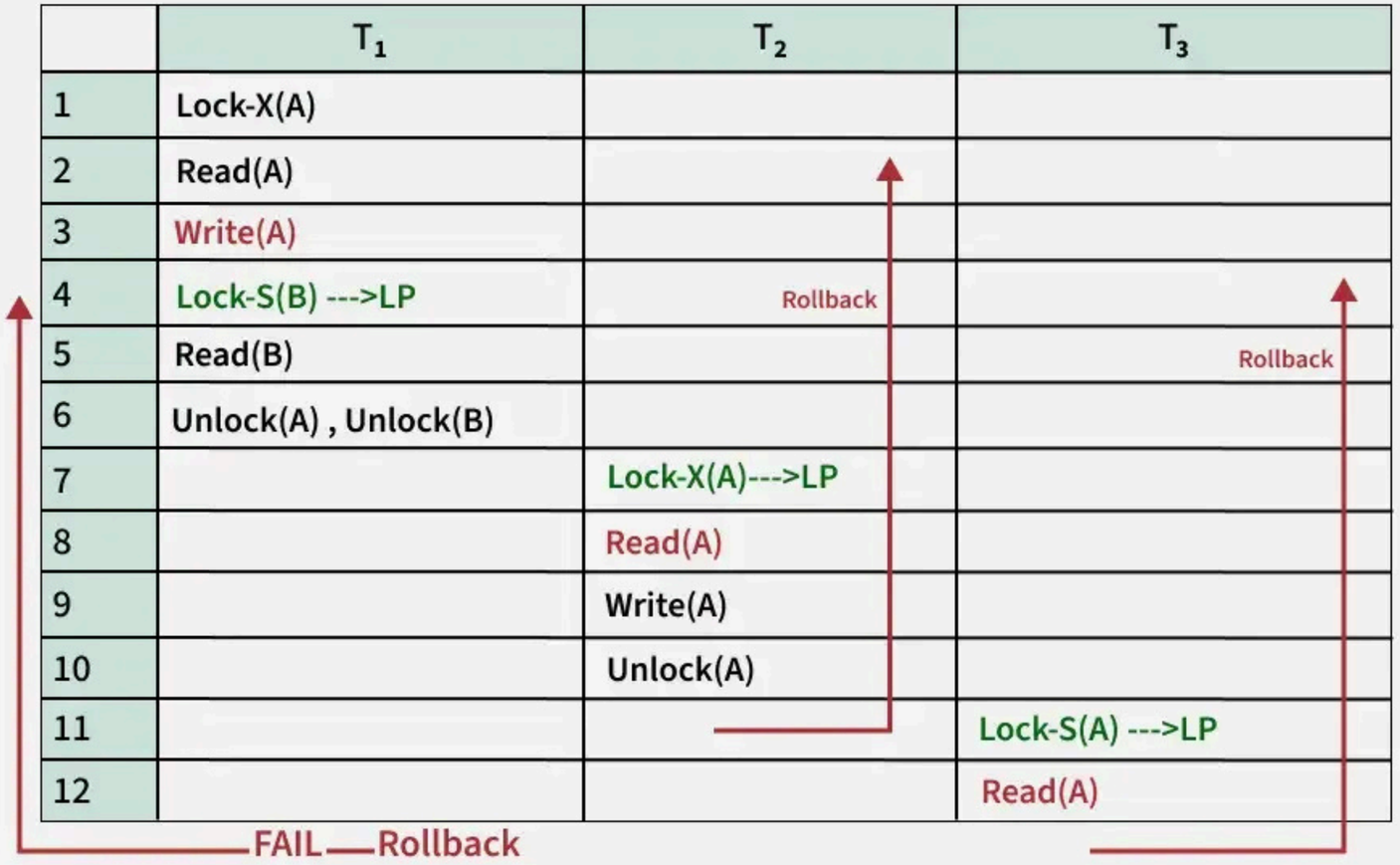
Shrinking Phase: The shrinking phase begins when the transaction releases its first lock. During this phase, it may release locks, but it cannot acquire or upgrade any new lock.

Lock Point: The lock point is the exact moment when a transaction obtains its final lock. After the lock point, it cannot acquire any additional locks, although it may continue its operations before releasing locks.

Upgrading a Lock: This means converting a shared lock (S) into an exclusive lock (X). It occurs when a transaction first reads data but later needs to modify it. The upgrade is only possible during the Growing Phase, when the transaction is still acquiring locks. Additionally, the upgrade is allowed only if no other transaction holds a shared lock on that data item.

Downgrading a Lock: This means converting an exclusive lock (X) into a shared lock (S). It happens when a transaction finishes modifying data and only needs to read it afterward. The downgrade must occur during the Shrinking Phase, when the transaction is releasing locks. Since this reduces restrictions, it is generally allowed.

EXAMPLE OF 2PL



LP - Lock Point

Read(A) in T_2 and T_3 denotes Dirty Read because of Write(A) in T_1 .

CATEGORIES OF TWO PHASE LOCKING

Strict Two-Phase Locking:

- Follows basic 2PL rules
- All X-locks held until commit/abort
- S-locks can be released earlier
- Prevents dirty reads, No cascading rollback

Conservative Two-Phase Locking:

- Transaction locks all required items before starting
- If all locks not available → transaction waits
- No partial locking
- Deadlock-free, ensures serializability but reduces concurrency, low practicality

Rigorous Two-Phase Locking:

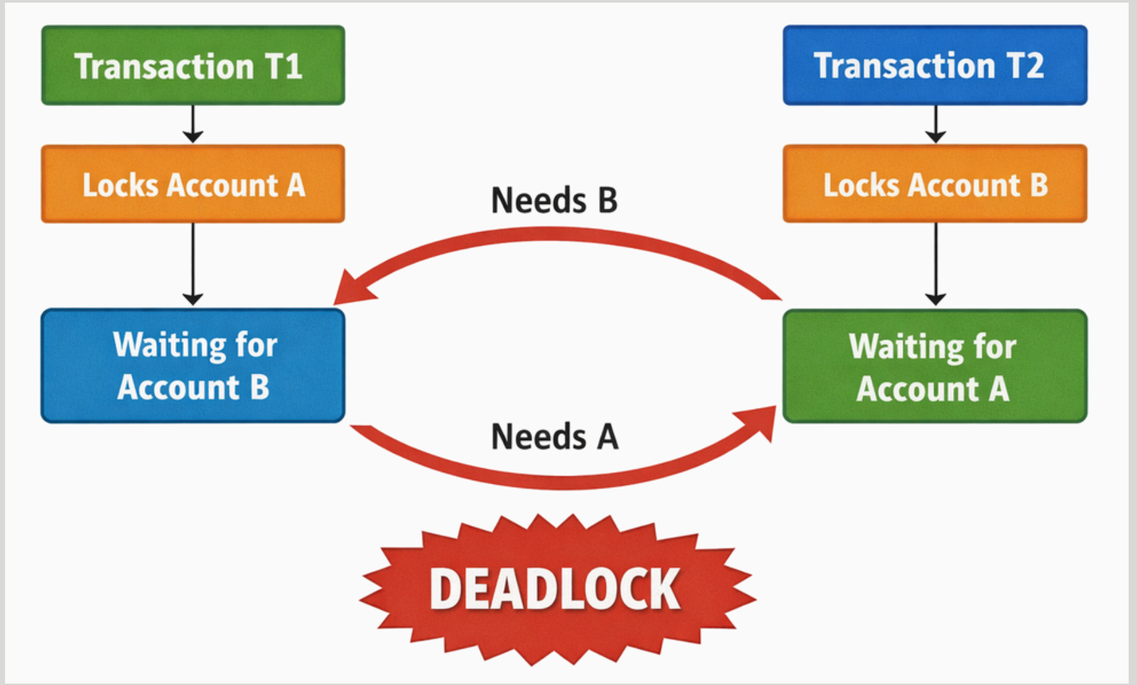
- Stronger than strict 2PL
- All locks (S + X) held until commit
- Locks released only after transaction finishes

Deadlock Handling

Deadlock is a state where transactions block each other permanently because each one is waiting for a resource locked by another transaction.

A very easy way to think about it is this:

- T1 says: “I have A, I need B.”
- T2 says: “I have B, I need A.”



Why Deadlocks Are a Problem?

- Transactions remain blocked and cannot complete their execution.
- Overall system performance drops because tasks are not progressing.
- Resources stay locked without being used effectively.
- In severe cases, the entire system may slow down or stop functioning properly.

Necessary Conditions of Deadlock:

- Hold and Wait: A transaction that already holds some resources can request more resources that are currently held by other transactions.
- No Preemption: A resource cannot be taken away forcefully from a transaction; it must be released by the transaction itself.
- Circular Wait: There is a cycle where each transaction is waiting for a resource held by another transaction in the chain.

How to Handle Deadlocks

Deadlock Avoidance: The idea is to design transactions so that deadlock does not happen.

- Follow the same order:
  - Always lock resources in a fixed order.
  - Example: always lock Account A first, then Account B.
  - Then two transactions will not block each other in opposite order.
- Use smaller locks and appropriate isolation levels:
  - For example, locking only specific rows instead of the whole table.
  - This reduces the chance of deadlock, but does not fully remove it.

Deadlock Detection: Here, the system allows deadlocks to happen but tries to detect them. If a transaction waits too long, the DBMS checks the situation.

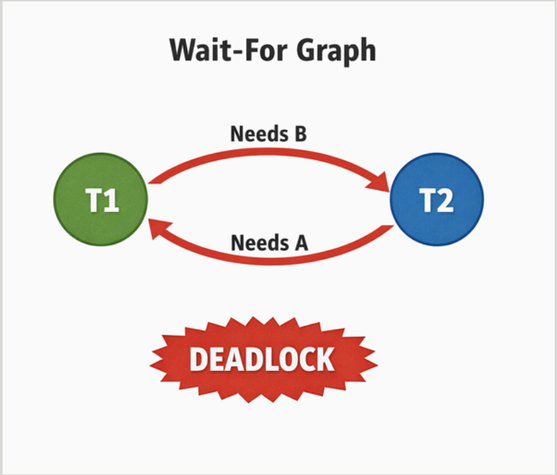
One of the Methods: Wait-For Graph

- Each transaction is a node
- An edge means one transaction is waiting for another

Example:

- T1 waits for T2 (needs Account B)
- T2 waits for T1 (needs Account A)

This creates a cycle → deadlock exists. This method is easier for small or medium systems.



How to Handle Deadlocks

Deadlock Prevention: The system stops deadlock from happening by controlling how resources are given. It checks transactions and does not allow situations that can create a cycle.
Lets' assume two transaction: T1(Older) and T2(Younger)

Wait-Die Scheme (Non-preemptive)

- T1 can wait if it wants T2's resource
- T2 is aborted and restarted if requests for resource held by T1.

Example:

- T1 (older) holds Account A
- T2 (younger) holds Account B

Case 1:

- T1 wants Account B → T1 waits

Case 2:

- T2 wants Account A → T2 is stopped and restarted

This avoids deadlock because younger transactions are not allowed to wait and form a cycle.

Wound-Wait Scheme (Preemptive)

- T1 can force T2 to stop
- T2 must wait if they want a resource held by T1.

Example:

- T1 (older) holds Account A
- T2 (younger) holds Account B

Case 1:

- T1 wants Account B → T2 is stopped, T1 continues

Case 2:

- T2 wants Account A → T2 waits

This avoids deadlock because older transactions are never blocked by younger ones.

Important Note: Aborted transactions restart with their original timestamp to reduce the possibility of starvation.

How to Handle Deadlocks

Deadlock Recovery: When a deadlock is detected, the system must break the cycle.

- Select one transaction as the **victim**
- Roll it back to release its locks
- Choose the victim with the **lowest rollback cost**

Types of Rollback:

- Total Rollback: Restart from the beginning
- Partial Rollback: Undo only enough to break the deadlock

Starvation Problem:

- A transaction may be selected repeatedly
- Increase its priority after each rollback

Solution:

- Track how many times a transaction is rolled back
- Increase its priority after each rollback
- Avoid selecting the same transaction repeatedly
- Use transaction age and rollback count when choosing a victim

Timeout-Based Handling:

- A transaction waits only for a fixed time
- If time is exceeded → transaction is rolled back

Advantages:

- Easy to implement
- Can break deadlocks

Limitations:

- May rollback even when there is no deadlock
- Choosing correct timeout value is difficult
- Starvation can still occur

Deadlocks cannot always be fully avoided
So systems use a combination of:

- Prevention
- Detection
- Recovery

Main goal: Keep the system running smoothly without transactions getting stuck forever

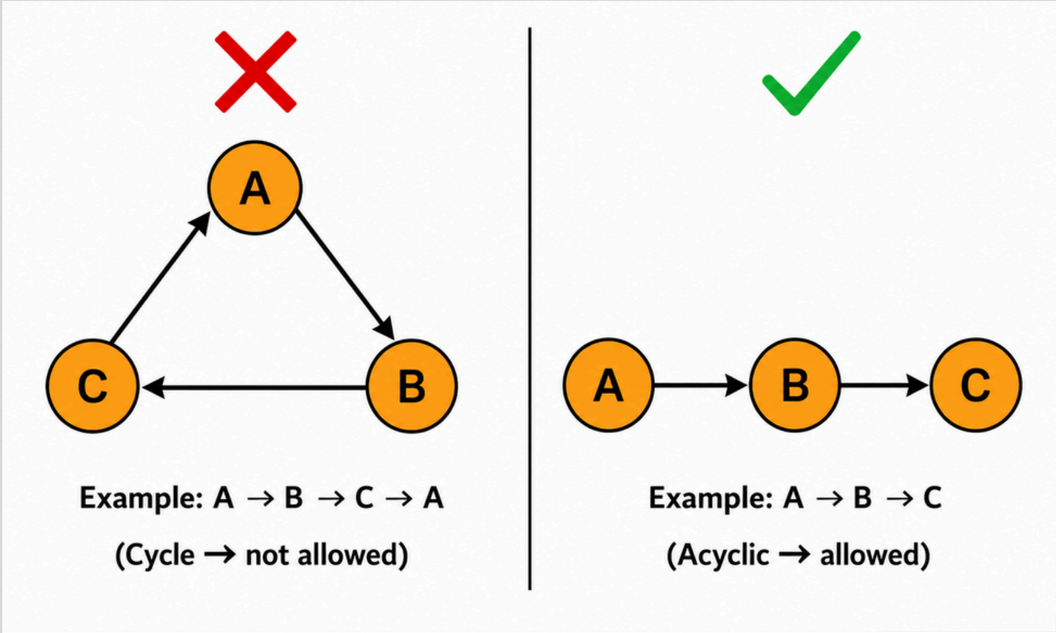
Graph Based Concurrency Control

Graph-based protocols are alternatives to Two-Phase Locking. They define a fixed order in which data items may be accessed.

The **Tree Protocol** a simple kind of graph protocol

Data items → Nodes
Access order → Directed edges

If A → B, a transaction accessing both items must access A before B.
The access graph must be acyclic.



Properties:

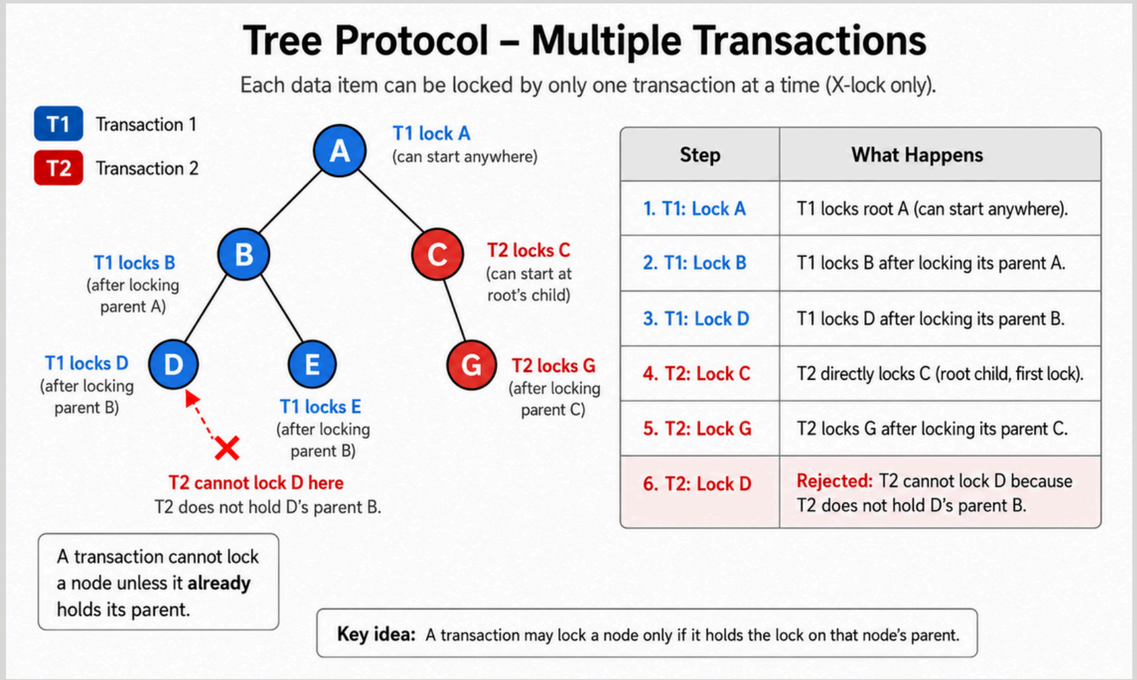
- **Conflict-serializable:** Following the predefined access order guarantees conflict serializability.
- **Deadlock-free:** The acyclic order prevents circular wait.
- **May cause cascading rollback:** Recoverability is not guaranteed.
- **Limitation:** The data-item access order must be defined in advance.

Tree Based Protocol

The Tree Protocol is a graph-based concurrency control method that guarantees conflict serializability and is deadlock-free. Unlike Two-Phase Locking (2PL), it enforces a hierarchical (tree-like) order when accessing data items.

Rules:

- **Only exclusive (X) locks are used:** Shared (S) locks are not allowed.
- **The first lock can be on any data item:** After that, a transaction can lock a data item only if it has already locked its parent.
- **Unlocking is allowed at any time:** There is no strict growing and shrinking phase like in 2PL.
- **No relocking is allowed:** Once a data item is unlocked, it cannot be locked again by the same transaction.



Step	What Happens
1. T1: Lock A	T1 locks root A (can start anywhere).
2. T1: Lock B	T1 locks B after locking its parent A.
3. T1: Lock D	T1 locks D after locking its parent B.
4. T2: Lock C	T2 directly locks C (root child, first lock).
5. T2: Lock G	T2 locks G after locking its parent C.
6. T2: Lock D	**Rejected:** T2 cannot lock D because T2 does not hold D's parent B.

What Next?

LECTURE 11:
QUERY PROCESSING



LOADING.....